

Managing AUV missions through small language models and behavior trees

Emilio Gigante

Abstract—Autonomous Underwater Vehicle (AUV) missions are constrained by limited onboard computation, harsh communication conditions, and strict energy and time budgets, which makes cloud-dependent Large Language Models (LLMs) unsuitable for mission management despite their strong reasoning capabilities. This thesis proposes a mission management approach based on a compact, fine-tuned Small Language Model (SLM) that enables operators to issue natural language commands, which are transformed into sequences of high-level tasks executed through behavior trees interfaced with the vehicle’s guidance system.

The SLM, instantiated as FLAN-T5-base and adapted via Low-Rank Adaptation (LoRA), is prompted with a structured combination of available operations, mission context derived from a dedicated mission memory, and the operator’s request, and is trained to output purely symbolic task tags that map directly to behavior-tree subtrees.

The method incorporates an explicit confidence estimation mechanism, adapted from the BSDetector framework, that combines Observed Consistency and Self-reflection Certainty into a scalar confidence score used to automatically trigger clarification dialogues when the model is uncertain, or to replan after execution failures by exploiting mission logs and failure context. The approach is first validated on a proof-of-concept virtual system inspired by the RAMI Marine Robots competition, where the SLM selects operations such as waypoint acquisition, gate traversal, buoy mapping, and pipeline inspection based on evolving mission knowledge. It is then integrated into the mission manager of a simulated AUV whose control stack is natively based on behavior trees, and evaluated using token- and sequence-level accuracy metrics and confidence statistics on dedicated training and testing datasets. Results show that the SLM-based manager achieves high sequence accuracy, correctly triggers the desired tasks, and assigns higher confidence to correct than to incorrect plans, while offering performance comparable to a rule-based mission manager but with substantially improved accessibility and usability for human operators under realistic resource constraints.

I. INTRODUCTION

Much of the recent research on autonomous vehicles focuses on the use of LLMs for operations management, including underwater vehicles, which, however, have known communication difficulties when submerged. In particular, only acoustic communication works, so the use of LLMs requires the vehicle to return to the surface to connect to the cloud. For this reason, it is important to explore the use of much smaller models that can be run on the vehicles’ on-board computers so that these tools can be used at every stage of the mission. These SLMs have limited capabilities but work well in understanding human intentions when properly trained, which is sufficient to build mission managers that use other mid-level control systems as support.

This work aims to:

- Build a system that generates an execution plan from a request in natural language.
- Ensure that the system is light enough to be deployed on an underwater vehicle.
- Ensure that the choice of system is influenced by the context of the mission.
- Ensure that the system expresses a level of confidence in the response it gives.

II. RECENT RESEARCH

Recent advances in applying language models to Autonomous Underwater Vehicle (AUV) mission management reveal both opportunities and challenges for embedded autonomous systems. Khorrambakht et al. [1] present a modular multi-agent framework leveraging large language models (LLMs) and reinforcement learning for coordinated multi-AUV operations. While their approach demonstrates scalable negotiation and task execution, it relies on resource-intensive large models unsuitable for embedded mission managers. Moreover, its strong focus on multi-agent coordination contrasts with this thesis’ emphasis on single-vehicle mission adaptability and failure recovery using smaller, efficient language models integrated with behavior trees.

Yang et al.’s OceanChat framework [2] effectively couples LLM-based natural language understanding with classical hierarchical planners, providing robust human-to-robot mission translation. The work’s introduction of photorealistic simulation validation is notable. However, its dependence on high-capacity LLMs and cloud connectivity limits applicability to fully autonomous underwater contexts, an area this thesis aims to address by optimizing for autonomy under communication constraints.

Chen et al. [3] propose FASTNav, which fine-tunes small language models (SLMs) for efficient on-board multi-point navigation using a teacher-student learning scheme. This approach aligns with the thesis goal of deploying resource-efficient models for embedded applications. Nonetheless, FASTNav targets structured indoor environments with point-to-point navigation and lacks mechanisms for environmental awareness or diversification of high-level actions critical to underwater mission complexity.

Finally, Word2Wave [4] offers a promising, user-friendly natural language interface for subsea mission programming based on a small T5 model. Its efficiency and simplicity strongly influenced this thesis design. Yet, the approach does not incorporate adaptive mission execution informed by environmental

feedback, highlighting a key limitation that this research seeks to overcome through integration of real-time adaptation and robust mission management.

Overall, these recent works illustrate the trade-offs between model scale, autonomy, and mission complexity relevant to underwater robotics. This thesis builds on their strengths while prioritizing compact language models, environment-aware behavior trees, and robust single-AUV mission autonomy suitable for edge deployment.

III. PROBLEM DESCRIPTION

Underwater missions are constrained by limited battery life, onboard computing power, thermal limitations, and slow movement, which demand careful mission planning and autonomy. Communication underwater is primarily acoustic, necessitating surfacing for mission updates, thus highlighting the need for onboard autonomous decision-making units that can replace human operators when plans fail.

SLMs, though less powerful than cloud-based large language models, present a feasible solution for onboard natural language mission management and supervisory tasks. These models can interpret commands, monitor mission progress, detect failures, and support adaptive replanning by integrating mission history and environmental feedback.

Language models are designed to return textual responses, which will therefore still be high-level commands. This makes them suitable for calling medium-level structures such as behavior trees, which have a simple and robust execution logic.

IV. FORMAL APPROACH

This section will provide a formal structure for the approach that will then be used in the following sections, first on a system designed specifically as proof of concept and then on a simulation system of a real robot.

The method must first be identified as a link between the operator and an execution system based on behavior trees, so it is best to start with a description of the latter to understand what is required of the system.

The article [5] describes briefly how these structures work.

A formal description of the method is provided in the next section.

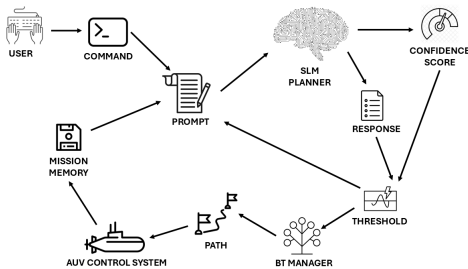


Fig. 1. When the user sends a command, it is combined with information from the mission memory to construct the prompt that is fed to the SLM, which returns the response along with a confidence coefficient. If this coefficient exceeds the threshold, the response is processed to trigger a BT; otherwise, it returns to the prompt and the user is asked to add further clarifying information.

A. Method

First, the **small language model (SLM)** is defined as a function that transforms a text **prompt** (P) into a **response** (R), also in text form.

$$SLM(P) = R \quad (1)$$

The response coincides with a string of text that corresponds to a list T_R of **tasks name tags** t_i that the system is set to handle.

From now on the symbol $\oplus$ will be used with the meaning of strings concatenation.

$$R = T_R = t_1 \oplus t_2 \oplus t_3 \oplus \dots \quad (2)$$

An example is provided to clarify the notation:

- $P = \text{"Close the valve"}$
- $t_1 = \text{"reach valve"}, t_2 = \text{"grab valve"}, t_3 = \text{"close valve"}$
- $T_R = \text{"reach valve, grab valve, close valve"}$

$$SLM(P) = T_R = t_1 \oplus t_2 \oplus t_3 \quad (3)$$

The confidence score, which is described in detail in section IV-D, serves to trigger, if lower than a certain threshold, a request for clarification to the model, in which the user must provide additional information beyond that which generated the uncertain response.

The structure of the prompt must be constructed following an analysis of the tasks that the vehicle can perform and the context in which it operates.

It is important that the prompt contains the minimum amount of information necessary to make one choice rather than another.

The planner is called upon in three situations that are structurally similar but with important differences.

As mentioned, a prompt is a text string, so every part of the prompt that will be mentioned later must also be a text string, so that the union of the parts corresponds to the concatenation of multiple text strings into one.

In general, the **prompt** (P) should contain a list of **all available tasks** (T_a), a description of the mission **context** (ctx), a **command** (cmd) which is the request that the user makes to the system in natural language, and a more technical **instruction** (ins) which instructs the model to select one or more of the available tasks and provides guidance on how to generate the response.

It should be noted that in the following equations is considered implicit, hence omitted, that each field has a name that defines it (as shown in example).

$$P = T_a \oplus ctx \oplus cmd \oplus ins \quad (4)$$

for example:

- $T_a = \text{"find pipeline, check pipeline, reach valve ..."}$
- $ctx = \text{"No pipeline found"}$
- $cmd = \text{"Detect the pipeline ID"}$
- $ins = \text{"Generate a comma separated tasks sequence from the available to satisfy the command"}$

$P =$ " Available tasks: find pipeline, check pipeline, reach valve ...

Context: No pipeline found

Command: Detect the pipeline ID

Instruction: Generate a comma separated tasks sequence from the available to satisfy the command

(Each field is appended in a new line, this is equivalent to the comma separation)

This is the main case, but if an uncertain response is generated from a prompt, a clarification mechanism is triggered in the system, which generates a **clarification prompt** (P_c) in which two elements are added to the previous prompt (P_p): one indicating the **previous response** (R_p) considered uncertain and the **clarification** (cla) field where the user is asked again to provide information.

$$P_c = P_p \oplus R_p \oplus cla \quad (5)$$

for example, let us use the previous example as P_p :

- $R_p =$ "check pipeline"
- $cla =$ "You have to find the pipeline first"
- $R =$ "find pipeline, check pipeline"

The other case occurs when one of the tasks initiated by a previous response fails. In this case, if the model is trained for this eventuality, it triggers automatic re-prompting (P_f) without requesting further information from the user, but using information about the task that has just failed, in particular through an element called **failure context** ($fctx$).

$$P_f = P_p \oplus fctx \quad (6)$$

This failure context involves defining which **task failed** (t_f); listing the **mission events** (E) appropriately retrieved from the mission logs and transformed into natural language so that they can be placed in chronological order as sentences; if the failed task was part of a sequence, **the tasks that remained in the queue** (t_q) will also be included, giving the model perspective on the phase in which the task failed.

$$fctx = t_f \oplus E \oplus t_q \quad (7)$$

for example:

- $R_p =$ "find pipeline, check pipeline"
- $fctx =$ "task 'find pipeline' failed. Mission events: survey started. pipeline not found. task failed. Remaining task: 'check pipeline' "
- $R =$ "find pipeline plan B, check pipeline"

It is important that there is a system that manages when the planner is triggered and that takes care of completing the command provided by the user with all the context elements useful for the model to make its choice, this will be called **manager**.

B. Model selection

The model chosen for the planner of the system described above is **FLAN-T5-base** [6].

T5 models [7] have proven highly effective for converting unstructured natural language mission descriptions into structured control sequences for marine robotics. The T5 architecture is based on the Transformer encoder-decoder model, which processes input text through embedding, self-attention, and feed-forward layers in the encoder and decoder with cross-attention mechanisms. [8]

For this work, FLAN-T5—a variant of T5 enhanced through instruction tuning [9], [6]—was chosen. FLAN-T5 improves generalization across diverse tasks and prompt formulations, supporting natural language command interpretation, mission context sensitivity, and interactive clarifications essential for autonomous mission management. Its strong zero-shot and few-shot capabilities reduce fine-tuning requirements while enabling versatile instruction following.

In comparative experiments, FLAN-T5 outperformed an equivalently sized LLaMa model [10], which showed tendencies to lose context and generate incorrect outputs despite training efforts. The smallest FLAN-T5 model was chosen initially for computational efficiency and inference speed, important in embedded AUV missions. However, its limited sensitivity to mission memory led to adopting FLAN-T5-base, which better integrates mission context, command variability, and confidence assessment. This base model forms the foundation for both theoretical and simulation implementations of the system. Further validations of FLAN-T5's classification capabilities in various domains reinforce its suitability for this application.

C. Model training

An initial dataset is constructed containing 100 examples for each batch (examples with the same output), which is split 80/20 for training and validation. In addition, a further dataset is constructed that is 10 times smaller and contains similar examples but which are more complex linguistically. Each batch contains classic and clarification examples with a ratio of 70/30, or failure and clarification examples with the same ratio.

Given the model and the dataset, focus shifts to fine-tuning, which is necessary because T5 and FLAN-T5 are pretrained on general corpora and do not fit the highly specific AUV mission management tasks without adaptation.

To efficiently fine-tune on the limited data available, the Low-Rank Adaptation (LoRA) technique [11] is employed, which modifies pre-trained weights by injecting trainable low-rank matrices, significantly reducing trainable parameters and computational cost while mitigating overfitting risks.

LoRA hyperparameters such as rank (r) and scaling factor (α) control adaptation size and learning rate, with dropout applied as regularization [12]. Training all layers with low rank is recommended [12], and in this work values suggested by LoRA literature are used, reducing trainable parameters from 248 million to about 5.9 million.

Training uses standard practices: splitting dataset into training and validation sets, monitoring training loss along with token accuracy [13] and sequence accuracy [14] on the validation set to select the best-performing model. Sequence accuracy

is most critical as the exact output sequence is required for correct mission execution.

Learning rate is linearly decayed to zero over epochs, and early stopping based on loss plateauing is applied to prevent overfitting [15]. This approach balances model adaptation quality with computational feasibility and robustness for deployment on embedded AUV systems.

D. Model confidence

The prompt engineering of the model has been highlighted as an area of focus, as it is a matter of great importance and is often the subject of research [16] [17].

However, no matter how much attention is paid to the construction of the prompt, the results may be inaccurate, inconsistent, incorrect and sometimes even nonsensical [18].

For this reason, an important feature that was chosen to implement in this work's system concerns the model's ability to return, together with the desired output, a value that reflects what we could in fact consider to be a degree of **confidence** in the response it is giving us based on its training.

The research [19] introduces the ironically named **BSDetector** method, which was adapted to this case study.

The method involves calculating two scores: Observed Consistency (OC) and Self-reflection Certainty (SRC), which are combined to obtain the confidence score (CS). This value is used to determine the degree of confidence that the system assigns to the response given by the model. This score is compared with a threshold, which determines whether the system trusts the response or not.

Let us look specifically at how the aforementioned scores are calculated.

Observed Consistency (OC): The command field (cmd) is extracted from the prompt (P) that generated the response we want to evaluate. A function (g_v) is then used which, through the model, generates a certain number of variants of the command ($cmd_{v,i}$) with a certain temperature value (T). It has been empirically assessed that seven variants are a sufficient sample to express a variety that is good proof of consistency.

$$g_v(cmd, T) = [cmd_{v,1}, \dots, cmd_{v,7}] = \vec{cmd}_v \quad (8)$$

Temperature is essentially a parameter that, when increased, allows the model to generate a sentence that is ideally identical in meaning but with a greater linguistic difference from the original. It should be noted that this parameter is highly sensitive, and that pushing it too far will result in sentences that not only differ in meaning from the original, but are sometimes meaningless [19].

Empirical tests were carried out to assess the ideal value of this parameter, eventually set at 0.9, in order to make sentences as different as possible while maintaining the same meaning. New prompts ($P_{v,i}$) are constructed replacing the original command in the original prompt with the command variants.

$$P(cmd_{v,i}) = P_{v,i} \quad \text{for } i = 1, 2, 3, 4, 5, 6, 7 \quad (9)$$

Each of the prompt variants is used to generate a response ($R_{v,i}$) through the model.

$$SLM(P_{v,i}) = R_{v,i} \quad \text{for } i = 1, \dots, 7 \quad (10)$$

Each of the responses generated from the variants is compared with the response to the original prompt. If the response generated by the variant matches the original response perfectly, it is assigned a full score, indicated by a variable (x_i), associated with the response, with a value of 1. If there is no exact match, this value is 0.

$$\text{if } R_{v,i} \equiv R \Rightarrow x_i = 1, \quad \text{else } x_i = 0 \quad \text{for } i = 1, \dots, 7 \quad (11)$$

An average of these values is then calculated by summing them and dividing by the total number. Once converted into a percentage, this value represents the score.

$$OC = \left(\sum_{k=1}^{n_v} x_i / n_v \right) \times 100 \quad \text{where } n_v = \dim(R_v) = 7 \quad (12)$$

This is, as the name of the score suggests, a good measure of the consistency of the model.

Self-reflection Certainty (SRC): Once the response has been generated, a certain number of variants of the command are created with very low temperature values, 0.3, similarly to how it was done in eq. 8. This is done in order to help the model in its self-evaluation, not varying too much from the original command.

It has been empirically assessed that five variants are sufficient to ensure that the model evaluates the command effectively.

After that, a number of prompts are created that are identical to the one that generated the response, but with the command replaced by its variants, like in eq. 9.

A scoring request (P_s) is added to each of the previous ones. This shows the previous prompt and the generated response asking the model to evaluate the response correctness w.r.t. the prompt with a digit: 0 if incorrect, 0.5 if not sure, 1 if correct.

$$P_{SRC,i} = P_{v,i} \oplus P_s \quad \text{for } i = 1, \dots, 5 \quad (13)$$

The model will return one of these three values for each of the prompt variants.

The summation of the responses represents the percentage that defines the score.

$$SLM(P_{SRC,i}) = R_{SRC,i} \equiv s_i \quad \text{where } s_i = 0 \wedge 0.5 \wedge 1 \quad (14)$$

$$SRC = \left(\sum_{k=1}^{n_v} s_i \right) \times 100 \quad \text{where } n_v = 5 \quad (15)$$

Confidence score (CS): The previously mentioned scores are combined simply through a single parameter β .

$$CS = \beta(OC) + (1 - \beta)(SRC) \quad (16)$$

Basically, one should rely on self-reflection certainty to the extent that one trusts the model's judgement.

Since the paper [19], which is based on GPT 3 and 4, uses a beta value of 0.7, meaning it does not place much trust in this capability, in this case one should trust this model even less, as it is considerably smaller in size. In fact, after conducting the necessary checks, it was noticed that the Self-reflection Certainty score is very unreliable, but it was still decided to give it a little importance in the final score to distinguish between cases with the same OC, so a beta value of 0.8 was set, making it less relevant. It is in some cases useful at least to see the score, it will be shown in the validation section.

Threshold

A threshold must be set in the system for the confidence score that determines the maximum amount of uncertainty we accept in order for the generated response to be accepted. In practice, if the CS is greater than the threshold, the response is processed directly for execution; if it is less than the threshold, the clarification mechanism is activated.

The threshold must be chosen as a trade-off between allowing the mission to run smoothly and taking a safety step by verifying responses defined by the threshold as uncertain.

If the threshold is too high, the user will often be asked to provide clarification for the response generated, making the command issuance phase tedious.

If the threshold is too low, potentially incorrect tasks could start executing immediately, forcing the user to abort the task and start over, which is a waste of time.

The correct approach would be to analyse the average score of incorrectly classified examples compared to correctly classified ones and set the threshold higher than the former. However, experiments carried out on the systems analysed in the following sections show that this is too cautious a measure, often forcing the user to intervene to clarify an output that is already correct. It was therefore decided to set the threshold at 50%, as this value aims at representing the correctness of the response, hence a value lower than 50% would mean that the response is more incorrect than correct.

V. PROOF-OF-CONCEPT VIRTUAL SYSTEM

A system has been artfully constructed in such a way as to test the approach seen in section IV.

In order to understand what type of task would be useful for the model, it was necessary to identify a known type of mission on which to base it.

For this purpose, it was decided to base the competition on the 2025 RAMI Marine Robots competition, in which the Ergo Team from the Department of Information Engineering at the University of Pisa recently participated.

The mission rulebook specifies actions to be performed autonomously by the vehicle, such as recognising the colours of buoys and performing a specific manoeuvre based on that colour, passing between two of them, finding markers on underwater structures, etc.

The approach was applied, taking care to use the prompt to select, through keywords, which parts of the context were actually useful for the choice, context that contains information

about mission development and objects found during it. The available operations were selected with a certain modularity that allowed the model to make similar but different choices based on the information in its possession.

Classic and clarification type of prompts are used, unlike the failure type, because the system is not designed to simulate a failure that could make sense to the model.

These training parameters were used:

- Number of epochs = 10
- Batch size = 8
- Gradient accumulation steps = 2
- Initial learning rate = $2.5 \cdot 10^{-4}$
- Warm-up steps = 50
- Early stopping patience = 3
- Early stopping threshold = 0.05

Which led to the following results:

Epoch (Step)	Training Loss	Validation Loss	Sequence Accuracy
1 (101)	11.1701	0.9470	0.3226
2 (202)	0.5075	0.0985	0.4888
3 (303)	0.1564	0.0568	0.6998
4 (404)	0.1106	0.0409	0.7916
5 (505)	0.0815	0.0265	0.8486
6 (606)	0.0678	0.0208	0.9057
7 (707)	0.0575	0.0199	0.9156
8 (808)	0.0500	0.0169	0.9181
9 (909)	0.0442	0.0154	0.9305
10 (1010)	0.0423	0.0153	0.9256

TABLE I

TRAINING AND VALIDATION METRICS PER EPOCH (1612 TRAINING EXAMPLES, 403 VALIDATION EXAMPLES).

To verify the effectiveness of the newly trained model, a dataset with 10 examples for each possible output was used, similar in form to the training dataset but with more complicated linguistic variations in the commands. The report of this test is provided below:

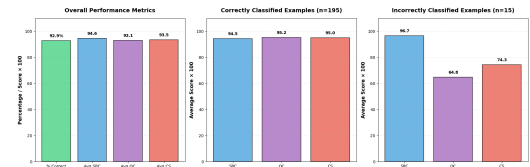


Fig. 2. Average test statistics on testing dataset

The sequence accuracy is consistent with that obtained during training, correctly classifying approximately 9 out of 10 examples. The average SRC and OC scores are quite high, indicating that the model has a good overall understanding of the general context of the mission.

The OC score is lower on average for incorrectly classified examples, but the same cannot be said for the SRC score, which is actually higher on average for incorrectly classified examples, making it unreliable. In general, there is a difference in the CS score that allows a reasonable threshold to be set. Incorrectly classified examples are still classified with answers similar to the correct one, which does not lead to serious execution errors.

These results will serve as a benchmark for those in section VI.

VI. APPLICATION OF THE FRAMEWORK ON THE AUV CONTROL SYSTEM

The approach discussed in the previous sections was applied to the control system developed by the ERGO team for the Zeno AUV owned by the Department of Information Engineering at the University of Pisa.

A. Vehicle control system

Zeno interfaces with the Robot Operating System (ROS) platform in its Melodic version. The system that determines the vehicle's movement is the **Guidance Manager** node, which receives information from the **Behavior Tree Manager** and transmits data to the **Path Follower** node or the Relative Guidance node. The latter was not used in the simulations described below. Each one of this nodes was designed by the ERGO team members.

The robot's position is determined by its latitude, longitude and depth coordinates. In terms of rotation, only yaw can be controlled, while roll and pitch remain fixed at zero. Descent and ascent are performed without these rotations, which led me to simplify the simulation by effectively creating a two-dimensional control, i.e. controlling only latitude, longitude and yaw.

The guidance system essentially manages two actions:

- **goal:** The vehicle reaches the target coordinates with the shortest possible route, i.e. a straight line, also adjusting the yaw to the desired value once it has been reached.
- **survey:** Coordinates are defined that the vehicle must reach, and the circular area that the vehicle must survey is centred at this point. A series of waypoints are generated that the vehicle must reach in sequence to complete the survey, which can be done starting from outside the area and following a transect path (also called a "lawn mower") or starting from the centre of the area with a spiral-like broken line.

All other actions that will be analysed in the following sections are combinations of these two and of simulated environment perception systems.

The role of the Guidance Manager is to receive instructions to initiate one of the two actions and generate waypoints which, once reached in sequence, determine the completion of the request.

The Path Follower aims to create a relative error instance with respect to the next waypoint to be reached, which is checked to zero by the vehicle.

The vehicle normally uses this relative error to determine the movement of a virtual joystick that replaces the one that should teleoperate Zeno via special software called Mission Manager. For simplicity, this work uses a node that represents Zeno's dynamic model and, receiving the relative error directly, controls it to zero, publishing the navigation status.

Zeno uses a camera on its nose to detect objects and is equipped with a neural network that recognises them in real time.

The simulated perception system was created in a very simplified manner, using 2D Stage simulation software, whereby simulated objects, in this case buoys, are defined only by their coordinates and colour. If the buoy is less than one metre from the vehicle and falls within its field of vision (i.e. within a 120° angle centred on its snout), the buoy is considered recognised, including its colour. These values were considered reasonable and consistent with some recent work in the field [20] [21].

B. Behavior tree manager

The Behavior Tree Manager receives instructions on the type of mission to be performed and the parameters, then uses the Guidance Manager to manage the execution of the mission through behavior trees.

The mission statement is provided in a message called **Set Info Mission Manager** (SIMM) containing the following information:

- mission tag: defines the type of mission to be performed, and therefore which main tree to trigger
- latitude, longitude, depth, yaw
- radius: when surveys are conducted, it defines the area
- yaw flag: defines whether a goal must also be achieved with the correct angular position
- spiral flag: defines whether the survey path should be constructed as a lawn mower or spiral pattern
- color flag: defines the target colour for those types of missions that search for a certain buoy or perform a certain action in relation to it

The available missions include going to a waypoint, passing between buoys, surveying, finding specific buoys, making a specific move for certain buoy.

During system execution, data is saved and then used by the mission manager:

- mission objects: defines which buoys have been found up to that point in the mission
- mission logs: all ROS environment logs useful for defining the status of the mission are saved here.

Since the ROS Melodic environment runs on an outdated version of Ubuntu, with respect to the ones which allow AI libraries to run, it was necessary to build a bridge node on which to exchange information on a more recent environment on which a language model can be run.

C. Rule-based usage of the system

This section describes how to use the system described above by interfacing directly with the behavior tree manager, without using the SLM mission manager, which will be covered in the next section.

As mentioned above, the BT manager requires message SIMM to start a mission. A configuration file has been created that associates a name tag with a unique message SIMM, so that once the system has been initialised, a programme can be used with the name tag of the desired mission as an argument, and this will send message SIMM to the BT manager, which will start the execution.

This system is simple and effective, but it should be noted that there is no assistance to the operator other than the information

that can be read from the mission terminals, so the operator must decide what to do on a case-by-case basis based on his assessment of the mission's progress.

D. SLM mission manager

The framework described in this section is the application of the manager element seen in section IV which uses the planner (the model) to send instructions to the vehicle control system seen previously.

The system consists of three elements:

- **chat:** the mean by which the user provides text for the system.
- **planner:** constructs the prompt and uses the SLM to generate the sequence of missions and the confidence score associated with each response.
- **manager:** receives information from the ROS environment, which it provides to the planner for the construction of prompts, receives prompt responses that define the mission that is sent to the ROS environment.

Generic missions and missions similar, as far as possible, to the RAMI rules have been defined:

- four missions to reach the north-west, north-east, south-west and south-east quadrants of the map
- four missions to survey the entire area of the same quadrants
- a mission that reaches a waypoint, which in the RAMI mission should be obtained externally
- a mission for crossing the gate
- a plan A and B to scan an area to find all the buoys, the reason for the double plan will be explained later
- a plan A and B to perform the corresponding move for each buoy

To improve the versatility of mission selection, the configuration file is reloaded each time the model is called by the manager, so that the user can change the parameters of a mission in real time. If necessary, also add a new mission with a new name tag.

The model is trained on certain examples that it may receive but for which the vehicle system is not configured, to return the word "skip", which is recognised by the manager as a command to do nothing, or, if present within a sequence, to move on to the next mission.

For example, when asked to parse the pipeline structure, the model, which does not have a mission among its options that can satisfy this request, would basically respond with a random mission, forcing the operator to stop operations, whereas in this way the model decides autonomously to respond to the request with inaction.

At this point, let's proceed by analysing the possible prompts that can be constructed for the model.

Let's start with the classic prompt (4):

- **Available missions:** all the name tags in the missions configuration file, plus 'skip'
- **Memory:** field that contains two subfields:
 - Buoys found so far: the colours of the buoys found up to that point in the mission

- Previous missions: list of missions successfully completed up to that point

- **Command:** given by the user
- **Instruction:** asks the model to provide one or more missions from the available ones

Here the memory field plays the role of context. In a competition such as RAMI, which takes place within a limited time frame, the memory of the missions carried out since the start of the session and the buoys found may also be superfluous. However, let us imagine a multi-robot control scenario, in which a single operator has to manage several AUVs. In this case, supervising each individual robot could prove difficult, and the robot's ability not to perform a mission that has already been carried out, perhaps even by another robot, could save time and resources. The same applies to objects found.

Therefore, it would be appropriate for the model to be trained to recognise the presence of the mission in the memory and to choose, depending on the mission planning upstream, a different mission, which may also be a plan B for the same mission, or to actually return the same mission to which the user can change the parameters.

Precisely for this latter case, there is a mechanism in the mission manager that verifies that the mission generated is not among those already completed. If the mission has already been completed, it creates a new prompt identical to the previous one, but adds two fields:

- **Previous response:** the previously generated response that matched the already completed mission.
- **Repeat mission:** input provided by the user which, if affirmative, effectively repeats the same mission, perhaps giving the user time to reflect and change the parameters. If negative, it returns the expected response in the case of training, which, as mentioned, may be a subsequent mission, a different mission, or a plan B for the previous one.

The system provides for uncertainty management with a function identical to that described in section IV-D, whereby when the model's confidence score is below the 50% threshold, the clarification mechanism is triggered, which constructs the clarification prompt (5):

- **Previous response:** Depends on the previous output, we want either to confirm or to change.
- **Clarification:** Has to provide either some variations of a confirmative sentence or some additional info that allows the model to change its response.

The reason why there is a plan B for 'map buoy area' and 'move' missions is that a failure management mechanism has also been implemented, which in both cases can occur, for example, when no buoy of the desired colour is found in the area where the survey was carried out. In this case, the system triggers an automatic reprompting mechanism that calls the model in question to decide what to do. The prompt in the event of failure (6) takes the form of the classical prompt plus a **failure context** field (7) that contains this information in the said example of the failed "find buoy" mission.

This provides us with information about the failed mission, the events that caused it to fail, and the remaining missions, which in turn provide information about the stage of the mission at which the queue was interrupted.

In this case, the model has been trained, for example, to respond with a plan B, which in practice consists of the same mission but in a different area of the map where we think those buoys are actually located. It should be noted that one could use an exact replica of plan A as plan B to be sure that there are no buoys in that area, and explore another area as plan C. One could also decide that if no buoys are found in that area, then move on to the next mission in the queue, for example, checking first that the other buoys are not there before changing area.

The manager shows for each completed mission the events that occurred during it to the user, but it is clear that these events are useful to the model only to the extent that this helps the model making the correct choice. In this case the mission events are not so numerous, so it is simpler to add all of them in the prompt. But the longer is the prompt, the more the model struggles, so one could think of using a language retrieval system [22] to select only those events that are important to allow the model to make the correct choice.

E. Validation

Here follow the training parameters used, found empirically through trial and error:

- Number of epochs = 7
- Batch size = 8
- Gradient accumulation steps = 2
- Initial learning rate = $3 \cdot 10^{-4}$
- Warm-up steps = 30
- Early stopping patience = 3
- Early stopping threshold = 0.05

Which led to these results:

Epoch (Step)	Training Loss	Validation Loss	Token Accuracy	Sequence Accuracy
1 (85)	14.6265	2.1338	0.9240	0.1294
2 (170)	0.8542	0.0832	0.9807	0.6735
3 (255)	0.1440	0.0309	0.9967	0.9412
4 (340)	0.0871	0.0235	0.9972	0.9500
5 (425)	0.0620	0.0183	0.9984	0.9706
6 (510)	0.0511	0.0161	0.9984	0.9706
7 (595)	0.0473	0.0161	0.9985	0.9735

TABLE II
TRAINING AND VALIDATION METRICS PER EPOCH
(1360 TRAINING EXAMPLES, 340 VALIDATION EXAMPLES).

Here is the report of this model test:

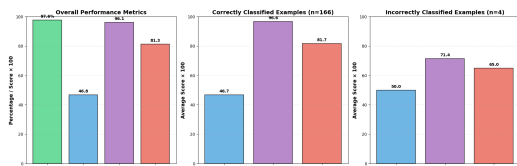


Fig. 3. Average test statistics on testing dataset

What is immediately evident is that, although the average Observed Consistency score is good, the Self-reflection Certainty score is very low, in particular it tells us that, in

the best case, the model is not sure if the response is correct, and on top of that the median score is zero, so the score is just bringing down the confidence score.

As mentioned previously in section IV-D, the SRC score is unreliable even when used on very large models. Once one realizes that the score is unreliable for one's purposes, one should simply abandon its use. This should certainly have been done in this case, but it wasn't only to demonstrate and justify this finding.

However, we saw in section V (which we will call PoC for brevity) how applying the same approach resulted in a significantly higher SRC score, in line with the OC score. This substantial difference with this system (which we will call R for brevity) may be due to several factors:

- Names in PoC are more self-descriptive, and make the model more confident that the sequence fits the prompt, while in R the names are tags that are linked to the BTs' mission type but do not clearly point to tasks at the linguistic level.
- Operations in PoC are more variegated and modular and their presence in the sequence is directly linked to the context, while in R most of the times for each request a single mission is the output and most of the time context is not used.
- The context in PoC is closely tied to the prompt and provides useful information in a much more descriptive and concise manner, helping the model make the right choices. In R, however, the context always has the same format for each prompt, which leads the model to give it less importance and therefore treat it more like noise.
- In PoC, a detailed analysis was done of the answers that the model had difficulty with, writing clarifying examples that confirmed correct answers and corrected incorrect answers. While in R the system has less difficulty giving the right answer, so mainly confirmatory clarifying examples were provided for a portion of the batches to build confidence, which led to an improvement from a practically zero SRC score to the current one.

Beyond this matter, I report below some considerations on the test results:

- Sequence accuracy is high, therefore most of the examples are correctly classified.
- The model does not make up any mission names, at worst it misclassifies the requests.
- The model, despite fine tuning, is reluctant to return the 'skip' output, probably this is seen by the model as a failure, which translated means that it does not fully satisfy its pre-training loss function.
- There are cases where, despite the instruction in the clarification field, or the similar repeat mission field, being clearly negative, the model returns the output it should have returned in the case of positive clarification. This could be due partly to the reason discussed above and partly to an unclear definition of the oppositional expression.

This tells us that the model is certainly effective in classi-

fyng, however its level of confidence in its answers could be significantly improved by optimizing the prompt with an automatic construction that is more effective in providing targeted information based on the request, so as to cause less confusion for the model. Furthermore, it would be good to provide the information in the most conversational way possible, and at the same time use more self-descriptive name tags that help the model self-evaluate.

Experiments were carried out on the simulation system described in section VI-A, evaluating the differences between the success rates of the same sessions performed using the rule-based system initially envisaged and the mission manager described in section VI-D.

Two types of experiments are conducted:

- Performance experiments that define the success statistics of the two systems in comparison with the operator who has full knowledge of how the system works.
- User accessibility experiments in which different operators are asked to use both systems to assess their ease of use.

Performance experiments: A session acts as a test for both the rule-based system and the SLM manager, as the latter works on top of the former, replacing the operator, so that any execution error is attributed to both. Despite this, the methods have almost identical success rates, showing that the manager maintains the previous functionality.

User accessibility experiments: Five operators who were unfamiliar with the system were asked to use the system with and without the SLM manager to verify its ease of use. The manager significantly reduced the number of actions required by the operator to complete tasks and also the total number of missions used, as well as errors due to misunderstanding of the assigned task.

Measurements show that, on average, the SLM manager consumes approximately 1.5 GB of RAM and 2% of the CPU of the machine used, as expected [7].

Deployment on the onboard computer on Zeno would therefore certainly be possible, but it would certainly require analysis of resource consumption by the control system and other systems, such as the neural network used for buoy recognition.

VII. CONCLUSIONS

The formal approach described in section IV has been adapted to the purpose-built system described in section V and to the real vehicle system described in section VI, succeeding in its aim of acting as a simplifying interface between the user and the execution system, in particular by managing to map natural language requests into operations useful to the latter.

There are some important considerations that need to be made, so that the application of the approach can be done as effectively as possible:

- The T5 architecture is text-to-text, so it's a good idea to make the information included in the prompt and the

names of the operations that will be output as conversational as possible, so as to facilitate understanding of the content of the request and the self-assessment that includes both the request and the generated response.

- The prompt should be constructed with as little content as possible to allow the model to make the correct choice, possibly filtering the prompt fields based on the request. Otherwise, the model may recognize some information as useless during training, leading it to ignore it even when it is actually useful for making one decision rather than another.
- When the execution system is designed to receive a limited number of operations, it's a good idea to assign each one a short, self-descriptive name and always present the model with options, along with instructions to choose between them. When these operations are a significant number and they would make the prompt too large it's better to avoid adding all the possible operations, but it's a good idea to train the model with responses that contain all the information needed to define a unique operation.
- A system of this kind is more effective if the operations are modular, since during training the model is able to better understand the function of that operation depending on the context, thus being able to generalise better.
- It's a good idea to run a few training attempts, so that with each attempt one can see which types of examples the model struggles with the most. From this information, one can more easily add clarifying examples to the training dataset. These help increase the confidence level of correctly classified examples that the model is unsure about, and correct incorrect examples using the clarifying format, even if the confidence level of the incorrect answer is high.
- Experiments suggest that the system using the SLM manager performs similarly to the rule-based system but is a much simpler and more user-friendly platform, which also reduces execution errors and therefore session time.

It can therefore be concluded that the approach is versatile and applicable to robot control systems, effectively replacing mission management systems. For its application, the most important phases involve analyzing potential requests, efficiently constructing the prompt, and simply and scalably building the training dataset.

A. Possible future improvements

This work primarily defines an approach aimed at creating an effective and efficient mission manager. An application has been described in section VI, but as described in the same section, this application is not intended to be ready-to-use for real-world missions. The idea is to lay the foundation for adapting the approach to each context, making the mission manager as useful as possible for any type of mission.

Possible continuations of this research could be found in the following areas:

- A survey of various language models of similar size to find the most suitable one for this purpose. Looking even

further, the construction of a specialized architecture to specifically handle this type of situation.

- An effective method for managing the construction of contextual elements starting from the mission environment, followed by an appropriate system for targeted selection of the information needed to satisfy the request presented to the system.
- A method for dividing system capabilities into operations that can be assigned a name tag, allowing the model to call the operations. These operations must be unique and have a clear name tag so that the model can understand how to construct sequences of operations based on context.
- Further analyze the relationship between these systems and behavior trees, possibly using more modular trees to replace the operations mentioned above.
- An optimized system for building datasets, both for training and testing, that distinguishes between different types of prompts and their ratio to the total number, also based on their meaning field.
- A data collection system suitable for the platform on which the experiments are performed, which makes it possible, in post-processing, to easily generate new examples that add functionality to the model, increase the model's confidence in correct answers, or correct errors.

REFERENCES

- [1] Ehsan Khorrambakht, Paolo Marinelli, Estelle Gerbier, Stefano Chiverini, Andrea Caiti, Paolo Di Lillo, and Andrea Munafò. Composable and modular autonomy for maritime robotics: Bridging human-robot collaboration, 2025. URL: <https://program-brest25.oceantechnical.org>.
- [2] Ruochu Yang, Mengxue Hou, Junkai Wang, and Fumin Zhang. Oceanchat: Piloting autonomous underwater vehicles in natural language, 2023. [arXiv:2309.16052](#).
- [3] Yuxuan Chen, Yixin Han, and Xiao Li. Fastnav: Fine-tuned adaptive small-language-models trained for multi-point robot navigation, 2024. [arXiv:2411.13262](#).
- [4] Ruo Chen, David Blow, Adnan Abdullah, and Md Jahidul Islam. Word2wave: Language driven mission programming for efficient subsea deployments of marine robots, 2025. [arXiv:2409.18405](#).
- [5] Michele Colledanchise and Petter Ögren. Behavior trees in robotics and ai: An introduction, 2022. [arXiv:1709.00084](#).
- [6] Hyung Won Chung, Le Hou, Shayne Longpre, et al. Scaling instruction-finetuned language models, 2022. [arXiv:2210.11416](#).
- [7] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2019. [arXiv:1910.10683](#).
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017. [arXiv:1706.03762](#).
- [9] Jason Wei, Maarten Bosma, Vincent Zhao, et al. Finetuned language models are zero-shot learners, 2022. [arXiv:2201.08239](#).
- [10] Nicholas Kluge Corrêa, Sophia Falk, Shiza Fatimah, Aniket Sen, and Nythamar de Oliveira. Teenytinyllama: Open-source tiny language models trained in brazilian portuguese, 2024. [arXiv:2401.16640](#).
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. [arXiv:2106.09685](#).
- [12] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. [arXiv:2305.14314](#).
- [13] Zhizhong Liu et al. A metric for token-level calibration on large language models, 2024. [arXiv:2406.11345](#).
- [14] DigitalOcean. Automated metrics for evaluating the quality of text generation, 2024. URL: <https://www.digitalocean.com/community/tutorials/automated-metrics-for-evaluating-generated-text>.
- [15] Garvesh Raskutti, Martin J. Wainwright, and Bin Yu. Early stopping and non-parametric regression: An optimal data-dependent regularization method, 2014. URL: <https://jmlr.csail.mit.edu/papers/volume15/raskutti14a/raskutti14a.pdf>.
- [16] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering for large language models. *Patterns*, 2025. In press. URL: <https://arxiv.org/abs/2310.14735>.
- [17] Dang Anh-Hoang et al. Survey and analysis of hallucinations in large language models. *Frontiers in Artificial Intelligence*, 8:1622292, 2025. URL: <https://www.frontiersin.org/articles/10.3389/frai.2025.1622292>, doi:10.3389/frai.2025.1622292.
- [18] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817*, 2024. URL: <https://arxiv.org/abs/2401.11817>.
- [19] Jiuhai Chen and Jonas Mueller. Quantifying uncertainty in answers from any language model and enhancing their trustworthiness, 2024. [arXiv:2308.16175](#).
- [20] Fei Kang, Dong Chen, Junjie Li, Gang Wan, and Zhe Li. Automated underwater concrete crack width measurement system based on dual lasers and deepcrack network. *Advanced Engineering Informatics*, 2025. Validates precise underwater measurements at 0.5-2 meter distances with millimeter accuracy. doi:10.1016/j.aei.2025.103713.
- [21] Guangliang Xu et al. Vision-based underwater target real-time detection for autonomous underwater vehicle. *Frontiers in Marine Science*, 10, 2023. Proposes lightweight CNN for AUV real-time detection with 0.053s computational time. doi:10.3389/fmars.2023.1175625.
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, and Tim Rocktäschel. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2020. [arXiv:2005.11401](#).